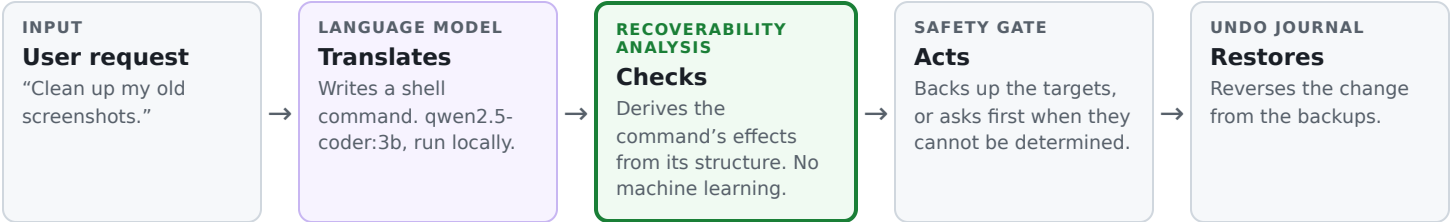


How the Recoverability Analysis Works

SynapseOS has two separate components. A local language model translates a request into a shell command; the recoverability analysis then checks that command before it runs: what it will change, and whether it can be undone. **Only the translation uses AI. The analysis itself does not use the language model:** it is deterministic program analysis that checks the model's output, specified in Chapter 3, §2.1(c) (pp. 62–66).

Where the analysis sits



■ **Probabilistic:** the same request may yield different commands. ■ **Deterministic:** the same command always yields the same decision.

Inside the analysis, in five steps

Running example: `find . -name "Screenshot*.png" -delete`

1	Parse Reads the command's structure with a shell-language parser.	Program <code>find</code> ; search path <code>.</code> ; filter <code>-name "Screenshot*.png"</code> ; action <code>-delete</code> .
2	Look up A rule table records what each program does to files.	<code>find ... -delete</code> removes every file that matches its filter; which files those are is not named in the command.
3	Compose Sees through searches, pipes, loops and wrappers to the program that acts.	The deletion is carried out by <code>find</code> itself on files it discovers while running. The same reasoning applies to <code>find -exec rm</code> , <code> xargs rm</code> , <code>for</code> loops and <code>sh -c</code> .
4	Resolve Runs only the proven read-only part, in a sandbox where nothing can be written.	ILLUSTRATION (SIMPLIFIED) OF AN INTERNAL STEP; USERS NEVER SEE IT: <pre>\$ find . -name "Screenshot*.png" # -delete replaced by a read-only print ./Screenshot 2026-09-20 184502.png ./Screenshot 2026-09-14 101211.png</pre>
5	Decide Classifies the effects and plans what to back up.	ACTUAL ANALYSIS OUTPUT, UNEDITED: <pre>command: find . -name "Screenshot*.png" -delete composition: enabled (full algorithm) class: recoverable-with-capture remove ./Screenshot 2026-09-20 184502.png remove ./Screenshot 2026-09-14 101211.png plan: what is captured before it runs trash ./Screenshot 2026-09-14 101211.png trash ./Screenshot 2026-09-20 184502.png</pre> Both files are backed up before the command runs: <i>trash</i> keeps a hard link to each, so the backup takes no extra space. Had their effects been undeterminable (an unknown program, inline Python, <code>shred</code>), the command would be classified <i>unrecoverable</i> and confirmation required.

Why it matters

It protects data, not intent The decision is made by code, not a model, so a wrong command that would destroy files is still caught. It cannot tell whether a command is what you meant: a wrong but harmless one runs. That part is only as good as the model.	What it improves on Coding agents check commands against a list of dangerous patterns and rely on git to undo mistakes. Personal files have no git, and a list misses targets hidden in <code>find</code> or <code>xargs</code> . On 777 test commands (§3.1(g)), a list let 41% of destructive ones run unprotected; this analysis let none, and backed up 89.5%.	It is the computer-science contribution The analysis is static program analysis in the tradition of abstract interpretation (pp. 25–26), not machine learning. The model is the interface; the analysis is the research.
---	--	--